Coding & Solution

Date	30 September 2026
Team ID	6ab3ec14b5170a9b58b3a658
Project Name	Implement Client Script and UI Policy
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/your-username/student-task-manager
Programming Language(s)	Python, JavaScript, HTML, CSS, SQL
Framework(s) Used	Flask (backend), SQLite (database), Bootstrap 5 (frontend)
Key Features Implemented	1. User registration and login with hashed passwords 2. Create, edit, delete and complete tasks 3. Task categories, priority levels and due dates 4. Search and filter tasks by status or priority 5. Dashboard showing pending and completed task counts 6. Input validation and user-friendly error messages 7. Responsive layout for desktop and mobile
Pending / Incomplete Features	1. Email reminders for upcoming deadlines 2. Task sharing between multiple users

	3. Unit test coverage for all modules
Setup / Run Instructions	1. Clone the repository: git clone <repository-url> 2. Move into the folder: cd student-task-manager 3. Create a virtual environment: python -m venv venv 4. Activate it: venv\Scripts\activate (Windows) or source venv/bin/activate (Mac/Linux) 5. Install dependencies: pip install -r requirements.txt 6. Initialise the database: flask init-db 7. Run the app: flask run 8. Open http://127.0.0.1:5000 in a browser

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	**Yes**
2	Meaningful variable and function names are used	**Yes**
3	Code includes comments / documentation where necessary	**Yes**
4	Error handling is implemented for critical operations	**Yes**
5	The application runs without critical errors	**Yes**
6	Code is committed to a version control repository	**Yes**

Additional Notes / Comments:

The application meets the core problem statement of helping students plan and track their academic tasks in one place.

The code is organised into separate modules (routes, models, forms, utilities) with descriptive names and inline comments for complex logic.

Error handling covers invalid input, failed database operations and unauthorised access, with clear messages shown to the user.

All code is committed to the repository with meaningful commit messages, and a README explains setup and usage.

Pending features (email reminders, task sharing and full test coverage) are planned for a future release.